Software Engineer — Take-Home Assignment

Staff Scheduling System

Build a small web application that helps a store manager plan weekly staff schedules from historical demand.

Version 1.1 • 29 October 2026

1. Overview

You are building a scheduling tool for the manager of a retail or food-service store. The manager needs to plan who works, and when, across a typical week. Staffing should be driven by how busy the store is at each hour — busier hours need more people on the floor — while respecting each staff member's contracted maximum hours and giving everyone a fair share of work.

This assignment is deliberately open-ended in places. Where a requirement leaves room for interpretation, make a reasonable decision, state your assumption briefly, and move on. We are as interested in how you reason about an ambiguous product problem as in the code itself.

Using AI tools is expected. This exercise assumes you will use Claude (or a comparable assistant) to help you build. We would like a short note on how you used it — what you delegated, what you verified, and where you overrode its suggestions.

2. What to Build

A web application with a user interface that supports the following. Each item below expands on the core requirement — read the clarifications, as they define the acceptance bar.

2.1 Create a schedule

- A user can create a schedule. A schedule is the top-level container for everything else below (staff, transactions, shifts, and the generated roster).
- A schedule represents one typical week, organised by day of week (Monday–Sunday) and by hour.

2.2 Add staff and their maximum weekly hours

- Within a schedule, the user can add staff members.
- Each staff member has, at minimum, a **name** and a **maximum number of work hours per week**.
- The user can add, edit, and remove staff.

2.3 Upload transaction (demand) data

- Within a schedule, the user can upload a transaction report that gives a transaction count for each hour of each day of the week.
- A sample file is provided (*report_Transaction_20260807_20260813.csv*) — download it here. Your importer should accept this format — see Section 3 for the layout.
- Treat the transaction count in an hour as a proxy for how busy the store is during that hour.

2.4 Define shifts

- Within a schedule, the user can create a set of shifts that a staff member can be assigned to. A shift is defined by only a **start time** and an **end time**.
- Seed the system with two default shifts: **07:00 – 15:00** and **15:00 – 23:00**. The user can add, edit, or remove shifts through a clear, usable interface (see 2.7).

2.5 Auto-schedule

- Provide an **Auto-schedule** button. When clicked, the system drafts a sensible weekly schedule automatically.
- The draft assigns staff to shifts across the days of the week so that busier hours (higher transaction counts) are covered by more staff than quieter hours.
- The draft must respect each staff member's maximum weekly hours and should aim to give every staff member a fair, useful amount of work — nobody scheduled for zero or near-zero hours while others are maxed out.
- The result is a draft, not a final answer: the user must be able to see it laid out clearly (by day, shift, and assigned staff) and adjust it afterwards. The quality of that view-and-edit experience matters — see 2.7.

2.6 Aggregated summary view

The system displays an aggregated table that lets the manager judge how well staffing matches demand. Broken down by day of week and hour, each cell shows three values:

- **Transactions** — the transaction count for that day and hour, from the uploaded data.
- **Staff hours** — the staff hours scheduled in that day and hour. Because each cell spans one clock hour, this equals the number of staff on shift during that hour (each contributes one staff-hour).
- **Transactions per staff hour** — transactions ÷ staff hours for that cell. Guard against division by zero (e.g. show "–" when no staff are scheduled).

The table also shows overall aggregations across the whole week:

- **Total staff hours** — the sum of staff hours across every cell (i.e. all scheduled hours in the week).
- **Total transactions** — the sum of transactions across every cell.
- **Transactions per staff hour (overall)** — total transactions ÷ total staff hours.
- **Average transactions per staff hour** — the mean of the per-cell "transactions per staff hour" values (over cells that have staff). Note this can differ from the overall figure above; show both.

How you lay this out is your choice (a single grid, stacked values per cell, or separate grids per metric are all fine). Illustrative figures for a single day, to fix the arithmetic:

Hour	Transactions	Staff hours	Txns / staff hour
12pm	33	3	11.0
1pm	48	3	16.0
2pm	33	2	16.5
Day total	**114**	**8**	**14.3**

(Illustrative only — staff-hour figures depend on the generated schedule. 33 + 48 + 33 = 114 transactions over 8 staff-hours gives 14.3 transactions per staff hour for the day.)

2.7 Viewing & editing the schedule and shifts

We care about the quality of the experience, not just that the actions are technically possible. Treat the view and edit flows as a design problem and make deliberate choices about layout and interaction.

- **Readable schedule view.** Present the weekly roster in a clear layout — by day and shift, showing who is assigned — that a non-technical manager can understand at a glance.
- **Direct editing of the schedule.** The user can adjust the roster directly: assign, reassign, and remove staff from shifts on any day. Editing should feel quick and obvious, not buried behind forms.
- **Managing shifts.** Creating, editing, and removing shifts should use an equally clear interface, and changes should flow through to the schedule and the aggregated view where relevant.
- **Live feedback.** When the user edits the schedule, the aggregated summary (2.6) should reflect the change. Surface problems clearly — for example, warn when an edit pushes a staff member over their maximum weekly hours or leaves an hour understaffed — rather than silently accepting or blocking them without explanation.

You have latitude on the interaction model (grid, drag-and-drop, inline controls, modals — your choice). We will look at the design judgement behind it, so favour an experience that is genuinely pleasant and efficient to use over one that merely exposes every field.

3. Provided Data

The sample file is a weekly transaction report. The first column is the hour of day (7am through 10pm). Each remaining column is a day, and each cell is the number of transactions recorded in that hour on that day. A trimmed sample:

Download the data: report_Transaction_20260807_20260813.csv (Google Drive)

Hour	Fri	Sat	Sun	Mon	Tue	Wed	Thu
7am	22	13	7	12	22	13	16
8am	25	44	32	32	35	33	45
9am	27	57	41	39	48	32	49
1pm	64	44	38	45	48	45	45
…	…	…	…	…	…	…	…
10pm	5	5	6	7	2	6	6

Notes: the sample week runs Fri 07 Aug – Thu 13 Aug 2026, but you should treat the columns generically as days of the week. Data covers 07:00–23:00, which lines up with the two default shifts. Your importer should tolerate reasonable variation (e.g. a different week label or ordering) and fail gracefully on malformed input.

4. Auto-schedule — Expectations

This is the heart of the exercise. There is no single correct algorithm; we are looking for a defensible approach and clear reasoning. A workable line of thinking:

1. **Estimate demand per hour.** Convert the transaction counts into a required number of staff for each hour — for example, one staff member per N transactions per hour, with a sensible minimum (such as one) whenever the store is open. You choose N and justify it.
2. **Map demand onto shifts.** Because staff are assigned to shifts (not individual hours), decide how many people each shift needs on each day so that the busiest hours within that shift are adequately covered.

3. **Assign staff to shifts.** Fill each day's shifts with staff while respecting every staff member's maximum weekly hours.
4. **Balance the load.** Distribute shifts so that everyone gets a fair amount of work rather than concentrating hours on a few people. Define what "enough hours" means (for instance, a minimum utilisation target, or balancing hours across the team) and apply it consistently.

Trade-offs are expected. Demand may exceed available staff hours, or vice versa. Handle these cases gracefully — for example, flag hours that are under-staffed, or leave capacity unused — and surface the outcome to the user rather than failing silently.

5. Technical Requirements

- **UI is required, and design counts.** The application must have a usable interface — this is not a command-line or API-only exercise. Treat viewing and editing the schedule and shifts as a design-sensitive problem (see 2.7), not merely a set of CRUD screens.
- **Stack is your choice.** Use whatever web stack you are most productive in. A single-page front end with a light backend (or even fully client-side) is fine; persistence can be in-memory, local storage, or a database — your call.
- **Keep it runnable.** We must be able to start it locally by following your README. Prefer a one- or two-command setup.
- **Scope sensibly.** Favour a clean, working end-to-end flow over half-finished breadth. Authentication, multi-user support, and deployment are explicitly out of scope unless you have spare time.

6. Deliverables

Item	What we expect
GitHub repository	A GitHub repo containing your source code plus any seed/sample data needed to demo the full flow end to end. Keep commit history intact — we like to see how the work evolved.
README	How to install and run; a short description of your approach to the auto-scheduler; and the assumptions you made.
AI-usage note	A few sentences on how you used Claude (or another assistant): what you delegated, what you checked, and where you disagreed with it.

Submission: push your work to a GitHub repository and share the link with the hiring contact by the agreed deadline. If the repo is private, add the contact(s) we provide as collaborators.

On-site presentation: in the final on-site interview you will present your solution and walk us through the code live. Expect to demo the running application, explain your auto-scheduling approach and key design decisions, and be ready to make or discuss small changes to the code on the spot. Please have the project running on your machine and the repository open.

7. How We Evaluate

Criterion	What we look for
Correctness	Does the full flow work? Does auto-schedule respect max hours and produce a plausible, demand-aware roster?

Criterion	What we look for
Problem reasoning	Quality of the modelling choices and how clearly assumptions and trade-offs are stated.
Code quality	Readable, reasonably structured, and maintainable. Tests where they add value.
User experience & design	The quality of the view/edit experience for schedules and shifts: can a non-technical manager read the roster at a glance and adjust it easily? Thoughtful layout and interaction design count here.
Communication	README and AI-usage note: concise, honest, and easy to follow.

8. Stretch Goals (optional)

Only if you have time and the core flow is solid:

- Let the user drag-and-drop or manually adjust assignments after auto-scheduling.
- Show a coverage view: for each hour, required vs. scheduled staff, highlighting gaps and overstaffing.
- Support per-staff availability or days off, and honour them in auto-schedule.
- Add roles/skills (e.g. a shift must include at least one supervisor).
- Export the finished roster (CSV or printable view).

9. Time & Logistics

- **Suggested effort:** roughly 4–6 hours. Do not gold-plate — if you run out of time, note what you would do next in the README.
- **Questions:** if something is genuinely blocking, reach out to [hiring contact]. Otherwise, prefer to make and document an assumption.

Good luck — we're looking forward to seeing what you build.